VLSI Design

Simon Dorrer
Johannes Kepler University
simon.dorrer@jku.at

2026-10-17

Table of contents

1 Introduction 2
1.1 Preamble on the Use of AI 2
1.2 Learning Objectives 3
1.3 Prerequisites 3
1.4 Course Content 3
1.5 Grading 4
1.6 Why ASICs? 4
1.7 A Taxonomy of ASICs 5
1.8 The Digital Design Flow 5
1.9 The IIC-OSIC-TOOLS Container 7
1.10 The AMS Chip Template 8
1.10.1 Flow Overview 8
1.10.2 Folder Structure and Makefiles 9
1.11 Agentic AI Chip Design 9
1.12 Example: The TinyWhisper SoC 10
1.12.1 Weak Signal Propagation Reporter 10
1.12.2 Chip Overview 11
1.13 Tools Installation 12
1.13.1 Local Installation 12
1.13.2 Containerized On-Demand Workspace System (COWS) 13
2 From VHDL to SystemVerilog 13
3 Modeling Guidelines for Synthesizable RTL 13
4 Functional Verification with cocotb 14
5 Sequential Elements and Static Timing Analysis 14
6 The Synthesis Library 14
7 Synthesis Optimizations 14
8 Design for Test (DFT) 14
9 Clock Domains and Metastability 14
10 The RTL-to-GDSII Flow with LibreLane 14
11 Capstone Project: A Small RISC-V SoC 15
12 Appendix: SystemVerilog Cheat-Sheet 15
13 Appendix: Tooling Cheat-Sheet 15
14 Appendix: Sample Exam Questions 15
Bibliography 15

1 Introduction

This is the material for an introductory-level course on digital **very-large-scale integration (VLSI)** design, held at JKU under course number 336.051 ("KV VLSI Design").

The course teaches the design of **digital application-specific integrated circuits (ASICs)** with a modern, fully **open-source** RTL-to-GDSII flow. We model hardware in **SystemVerilog**, verify it with **cocotb** (Python), inspect waveforms in **GTKWave**/**Surfer**, and harden, assemble and tape out a complete chip — including the padframe — with **LibreLane** on the **IHP SG13G2** 130 nm open PDK. In the course project, you take your own **RISC-V SoC** all the way to a tapeout-ready chip.

> ! Important
>
> All course material (the source of this document, the SystemVerilog code, the cocotb testbenches, the LibreLane configurations, and the Python sources for all figures) is made publicly available on GitHub (follow this link) and shared under the Apache-2.0 license.
>
> Please feel free to submit pull requests on GitHub to fix errors and omissions!

> ! Important
>
> Please use the IIC-OSIC-TOOLS image with tag `2026.09` or later. It contains all tools and the PDK used in this course.

1.1 Preamble on the Use of AI

Artificial intelligence offers impressive possibilities today: it can explain how things are connected, provide examples, help with debugging, or point out different ways to solve a problem. Precisely for this reason, however, there is a risk of relying too heavily on such tools while learning, and thereby not practicing enough of exactly those skills that this course is meant to build.

In the written exam, you will have to solve problems without any technical aids. There, the only thing that counts is which concepts, methods, and skills you have mastered yourself. Likewise, in the oral presentation of your project, you have to be able to explain what you did and why you did it.

If you want to complete a long-distance run in a few months, you have to build up your own endurance. You could, of course, cover your weekly training route faster on an e-scooter, but that does not train your ability to run the distance yourself. The same holds for your studies: if AI regularly does the actual thinking for you, you will not sufficiently build up the skills you need.

So feel free to use AI as a learning partner, but not as a substitute for your own thinking. Always try to understand and solve a problem on your own first. Afterwards, use AI, for example, to check your solution, to learn about an alternative approach, or to have a difficult

topic explained to you. The goal of this course is not to arrive at a correct answer as quickly as possible, but to develop the competence to get there on your own.

In the end, you are here to learn something, so do not cheat yourself! This matters beyond the course, too: as AI agents take over more and more routine design work (see Section 1.11), engineers who know how to prompt these tools and, above all, how to critically review their results will always be needed. And you can only review what you understand yourself.

1.2 Learning Objectives

After completing this course, you will be able to:

- work with the **IIC-OSIC-TOOLS** container and manage your designs with **Git** and **GitHub**;
- read and write synthesizable **SystemVerilog** RTL for synchronous digital designs, and translate idiomatic VHDL into idiomatic SystemVerilog;
- apply **modeling guidelines** that lead to clean, fast, and testable synthesis results (avoiding latches, inferring proper registers, structuring arithmetic);
- verify RTL with **self-checking cocotb testbenches** and reason about coverage and corner cases;
- understand and explain the **RTL-to-GDSII flow** (synthesis, floorplanning, placement, clock tree synthesis, routing, signoff) and drive it with **LibreLane**;
- analyze timing using the concepts of **setup/hold, propagation delay, clock skew, and static timing analysis (STA)**;
- explain **design for test (DFT)**, the stuck-at fault model, scan chains, and test coverage;
- handle **multiple clock domains** safely with synchronizers and reason about metastability;
- integrate a digital macro into a **padframe** and assemble a complete, **tapeout-ready ASIC** with open-source tools;
- design, verify, harden, and document a **RISC-V SoC with your own extension** as a capstone project;
- optionally, use **LLM agents** efficiently, and critically, in a chip design workflow.

1.3 Prerequisites

The course is an elective for students of the Bachelor's program in Computer Science and of the Master's program in Electronics and Information Technology (ELIT). To follow it, you should bring:

- a basic understanding of CMOS transistors;
- a good understanding of digital logic (gates, flip-flops, etc.);
- basic experience with a hardware description language (e.g., VHDL);
- your own laptop with at least 20 GB of free disk space for VS Code, Git, and the IIC-OSIC-TOOLS (see Section 1.13).

The following JKU courses, whose material is also openly available, complement this course well but are not required:

- **KV Analog Circuit Design**: https://github.com/iic-jku/analog-circuit-design
- **KV Design of Complex Integrated Circuits**: https://github.com/iic-jku/design-complex-ic

1.4 Course Content

After this introduction, the course covers the following topics:

1. from VHDL to SystemVerilog, and modeling guidelines for synthesizable RTL (Section 2, Section 3);
2. functional verification with cocotb (Section 4);
3. sequential elements and static timing analysis (Section 5);
4. the synthesis library and synthesis optimizations (Section 6, Section 7);
5. design for test and clock domain crossing (Section 8, Section 9);
6. the RTL-to-GDSII flow with LibreLane, chip assembly, and padframe (Section 10);
7. the project: a tapeout-ready RISC-V SoC (Section 11).

The appendices provide a SystemVerilog cheat-sheet (Section 12), a tooling cheat-sheet (Section 13), and sample exam questions (Section 14).

> **i** Further Reading
>
> These notes are self-contained but kept concise. For depth we recommend:
>
> - [1] for digital design and RISC-V computer architecture.
> - [2] for CMOS VLSI design from circuits to systems.
> - [3] for clock-domain-crossing techniques in SystemVerilog.

1.5 Grading

The grade consists of two equally weighted parts:

- **Part 1 — Theory (50%)**: The theoretical lectures are mixed with small examples. This part is assessed by a **written exam before Christmas**; the exact date is announced in the lecture.
- **Part 2 — Project (50%)**: You implement a complete, **tapeout-ready RISC-V CPU** plus your **own extension** (analog or digital) and present your work in an **oral presentation** (Section 11).

> **Tip**
>
> The best project might actually be taped out! Moreover, the project can be extended into a seminar work (+3 ECTS) or a thesis.

1.6 Why ASICs?

Application-specific integrated circuits are designed when standard, off-the-shelf components cannot meet the requirements. Typical motivations are:

- **Specification**: performance, power, or size targets unreachable with standard parts.
- **Integration**: combining digital logic, analog circuits, and on-chip memory into a single mixed-signal chip.
- **Economics**: at high volume the per-unit cost drops dramatically, and integrating more functionality into one chip reduces system cost.

Historically, the cost *per transistor* has fallen by orders of magnitude as integration density doubled roughly every 18–24 months (**Moore's law**). What was once only affordable to large companies is, with open-source PDKs and tools, now accessible to students and researchers.

1.7 A Taxonomy of ASICs

ASIC implementation styles trade off non-recurring engineering (NRE) cost, unit cost, performance, and design effort:

Table 1: ASIC implementation styles

Style	Masks	Effort	Typical use
PLD / FPGA	none (field-programmable)	low	prototypes, low volume
Gate array / sea-of-gates	only wiring layers	medium	medium volume
Standard-cell IC (CBIC)	all layers	medium–high	the workhorse of digital design
Full-custom IC	all layers, hand-crafted	very high	highest performance / lowest unit cost

This course focuses on the **standard-cell** style: we describe behavior in SystemVerilog and let the synthesis tool map it onto a library of pre-characterized standard cells from the SG13G2 PDK.

1.8 The Digital Design Flow

The core of this course is the digital design flow that turns a hardware description into a manufacturable layout. Since this layout is traditionally handed over to the foundry in the GDSII file format, it is called the **RTL-to-GDSII** (or simply RTL-to-GDS) flow. Figure 1 shows its main steps.

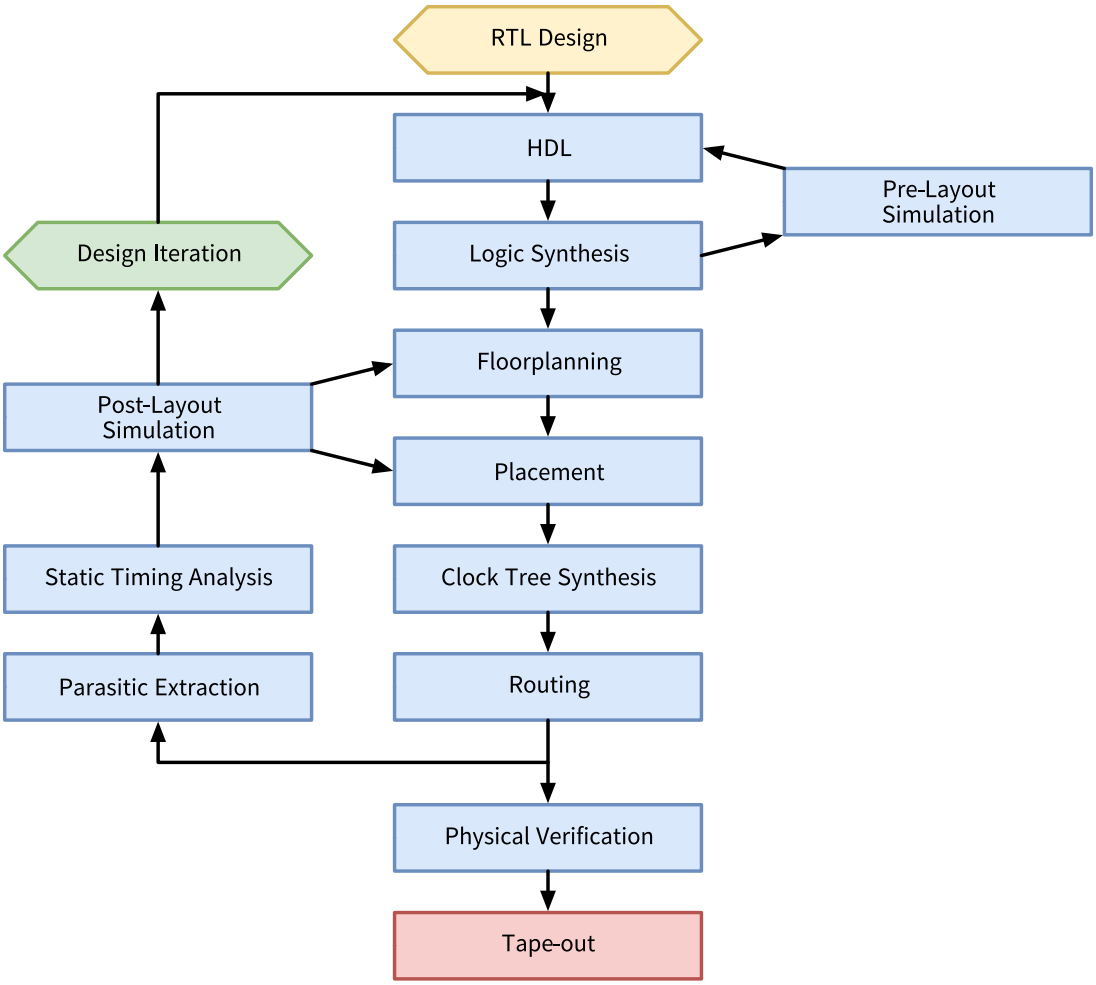


Figure 1: The RTL-to-GDSII design flow. The synthesized gate-level netlist is checked by pre-layout simulation. After routing, parasitic extraction, static timing analysis, and post-layout simulation feed back to floorplanning and placement or, if necessary, to the RTL (design iteration). Only a design that passes physical verification is taped out.

The flow starts with the **RTL design**, described in a hardware description language (HDL) such as SystemVerilog. **Logic synthesis** translates the RTL into a gate-level netlist of standard cells of the target technology, guided by constraints such as the clock frequency. Long before any layout exists, the design is simulated extensively: first at RTL, and then as a synthesized netlist (**pre-layout simulation**). Every bug found here leads back to the HDL.

The physical implementation then turns the netlist into a layout. **Floorplanning** defines the size of the die and the core, the placement of I/Os and macros, and the power grid. **Placement** positions the standard cells, **clock tree synthesis (CTS)** builds a balanced clock distribution network, and **routing** connects all cells with metal wires.

Only after routing are the resistances and capacitances of the wires known. **Parasitic extraction** annotates them to the netlist, on which **static timing analysis (STA)** and **post-layout simulation** check whether the design still meets its timing and functional specification. Violations are fixed by revisiting floorplanning and placement or, if this is not enough, by changing the RTL itself (**design iteration**), e.g., by pipelining a critical path. Finally, **physical**

verification checks the layout against the design rules of the foundry (design rule check, DRC) and against the netlist (layout versus schematic, LVS). Only a design that passes all checks goes to **tape-out**, i.e., is sent to the foundry for fabrication.

Besides these steps, **design for test** inserts scan chains so that every fabricated chip can be tested (Section 8), and designs with several clocks need safe **clock domain crossings** (Section 9). For a complete chip, the hardened digital macro is finally integrated into a **padframe** and assembled into the chip top level (Section 10). Table 2 lists the open-source tools used in this course for each step and where the step is covered.

Table 2: Steps of the RTL-to-GDSII flow, the open-source tools used for them in this course, and where they are covered. LibreLane orchestrates all steps from synthesis to GDSII; the PDK is IHP SG13G2 (130 nm).

Flow step	Open-source tool	Covered in
HDL	SystemVerilog	Section 2, Section 3
RTL, pre- and post-layout simulation	cocotb with Icarus Verilog or Verilator; GTKWave, Surfer	Section 4
Logic synthesis	Yosys	Section 6, Section 7
Floorplanning, placement, CTS, routing	OpenROAD	Section 10
Parasitic extraction	OpenRCX	Section 10
Static timing analysis	OpenSTA	Section 5, Section 10
Physical verification	Magic, KLayout (DRC); Netgen, KLayout (LVS)	Section 10
Chip assembly and padframe	LibreLane with the AMS chip template	Section 10, Section 11

1.9 The IIC-OSIC-TOOLS Container

The legacy version of this course relied on commercial tools (Vivado, ModelSim, Synopsys Design Compiler, Cadence Innovus, TetraMAX, PrimeTime). This edition is built entirely on open-source tools, all pre-installed in the **IIC-OSIC-TOOLS** container [4]. This all-in-one Docker/Podman image for analog and digital IC design is curated at JKU and runs natively on `x86_64` and `arm64` machines (e.g., Apple Silicon). Since everyone uses the identical container, results are reproducible, and "works on my machine" problems disappear.

As Table 3 shows, the container covers much more than this course needs: full-custom (analog) design with schematic entry, analog simulation, layout, and electromagnetic (EM) simulation, semi-custom (digital) design with simulation, synthesis, and complete RTL-to-GDSII flows, as well as several open-source PDKs. In this course, we mainly use the digital tools (Table 2) with the IHP SG13G2 PDK; the analog tools appear in the AMS chip template (Section 1.10).

Table 3: A selection of the tools and PDKs in the IIC-OSIC-TOOLS container

Category	Tools and PDKs
Schematic entry	Xschem, Qucs-S
Analog simulation	ngspice, Xyce, VACASK, CACE
Layout editing	Magic, KLayout, GDSFactory
Digital simulation and waveforms	Icarus Verilog, Verilator, cocotb, GTKWave, Surfer
Logic synthesis	Yosys (with the slang and GHDL front ends), ABC
RTL-to-GDSII flows	LibreLane, OpenROAD Flow Scripts, SiliconCompiler, QFlow
Place and route, STA, extraction	OpenROAD, OpenSTA, OpenRCX
DRC, LVS, and PEX	Magic, KLayout, Netgen, KLayout-PEX
EM simulation	Palace, openEMS
PDKs	sky130A, gf180mcuD, ihp-sg13g2, ihp-sg13cmos5l

1.10 The AMS Chip Template

Hardening a single digital block is only part of the way to a chip. A tapeout-ready chip additionally needs a padframe and a seal ring, the placement and wiring of all macros on the top level, logo and metal-fill insertion, full-chip verification, and finally a bonding diagram for packaging. To get there without fighting the tools, this course builds on the **ihp-sg13g2-ams-chip-template** [5], an open-source GitHub template repository for analog mixed-signal (AMS) chips developed at JKU:

- It is released under the permissive **Solderpad Hardware License v2.1**.
- Create your own repository with GitHub's *Use this template* button (or simply `git clone` it), enter the IIC-OSIC-TOOLS container, and run `make all`. The result is a **tapeout-ready AMS chip**.
- A step-by-step **tutorial**, written in Quarto, walks through all IC design steps on IHP's 130 nm process, with simple exercises to learn by doing. You start your project with this tutorial (Section 11).
- **Regression tests** via GitHub Actions ensure that the flow keeps working with every new release of the tools.

The template is also available for the ihp-sg13cmos5l, sky130A, and gf180mcuD open PDKs, and several chips have already been taped out with it, including TinyWhisper (Section 1.12).

1.10.1 Flow Overview

The Makefile-driven flow of the template covers:

- simulation, linting, and hardening of the RTL;
- layout checks and reports of the hardened digital macro;
- simulation of all analog testbenches, including process, voltage, and temperature (PVT) corners and Monte Carlo (MC) analysis;

- layout checks of the analog and digital blocks (DRC, LVS, and parasitic extraction, PEX);
- digital-on-top assembly of the chip top level and its layout check (DRC), including
  - seal ring and padframe generation,
  - macro placement and wiring, also with non-default rules (NDRs), and
  - logo and fill insertion;
- full top-level LVS and simulation without black-boxing;
- generation of the bonding diagram.

1.10.2 Folder Structure and Makefiles

The folder structure of the template was discussed with the FOSSi community. Every macro in `macros/` (e.g., the digital `counter` and the analog `inverter` examples) follows the same scheme as the chip top level, with its own `Makefile` and folders such as `rtl/`, `testbenches/`, `schematic/`, `layout/`, `netlist/`, `verification/`, and `final/`. Since a macro can itself contain macros, the structure is recursive. This makes it easy to share macros between projects and to reproduce the results of others.

At the top level, `make all` runs four steps:

1. `make build-all` builds the bondpad, the logos, all enabled macros, and finally the chip top level (LibreLane assembly, reports, GDS, netlists, logo and fill insertion, and renders);
2. `make magic-drc` (or `make klayout-drc`) checks the final layout of the chip;
3. `make sim-all` simulates the chip (RTL and gate-level simulation with cocotb, gate-level simulation with Xschem);
4. `make bondplan` generates the bonding diagram for packaging.

Targets that build an IP block or a macro recursively call the `Makefile` of that block.

1.11 Agentic AI Chip Design

Large language model (LLM) agents are increasingly able to operate a complete chip design environment: they do not only answer questions, but also edit design files, run tools, interpret the results, and iterate. Figure 2 shows such an agentic workflow.

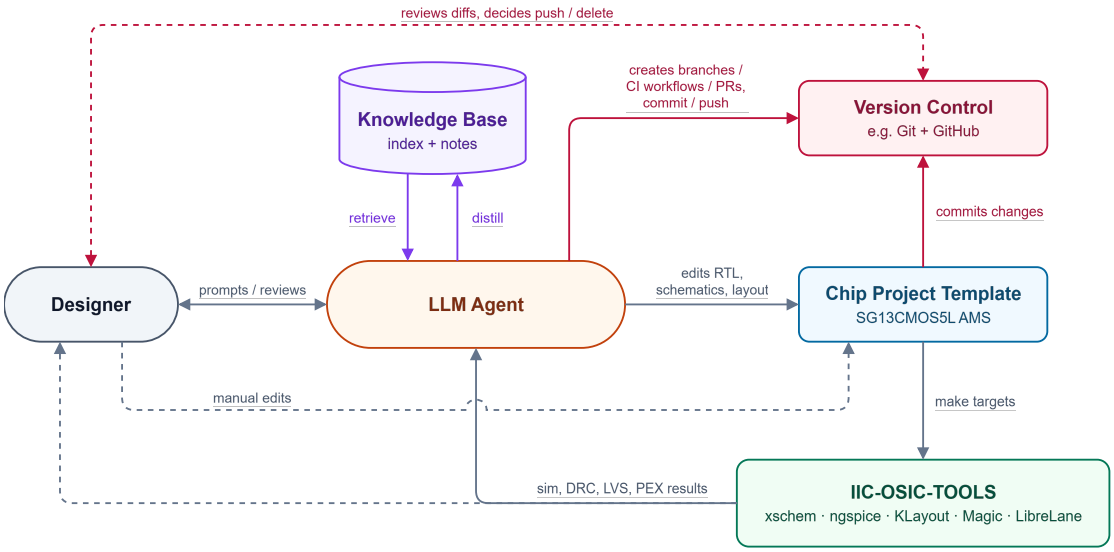


Figure 2: Agentic AI chip design workflow: the designer prompts and reviews an LLM agent, which edits the chip project, runs the tools via make targets, and tracks all changes in version control (shown here for the SG13CMOS5L version of the AMS chip template).

The designer interacts with the LLM agent through prompts and reviews. The agent edits the RTL, schematics, and layout of a chip project based on the AMS chip template (Section 1.10) and runs its make targets, which call the tools of the IIC-OSIC-TOOLS. The results of simulation, DRC, LVS, and PEX flow back to the agent and to the designer. A knowledge base (an index plus notes) lets the agent retrieve relevant information and distill new insights for later sessions. All changes are tracked in version control (e.g., Git and GitHub): the agent creates branches, commits and pushes changes, sets up CI workflows, and opens pull requests, while the designer reviews the diffs and decides whether changes are pushed or discarded. Of course, the designer can still edit the project manually at any time.

Such a workflow is only possible because every step of the flow is scripted, every tool runs in a reproducible container, and every result is machine-readable. Note the role of the designer: they remain responsible for the design and have to judge whether the results of the agent are correct, which brings us back to Section 1.1. Using LLM agents efficiently is an optional learning objective of this course.

1.12 Example: The TinyWhisper SoC

What is possible with open-source tools and the AMS chip template is demonstrated by **TinyWhisper**, an open-source, fully integrated, multi-mode short-wave transmitter for amateur radio, developed in a cooperation between JMU Würzburg and JKU Linz. TinyWhisper combines a RISC-V CPU with peripherals, custom digital signal generation, and an analog front-end on one chip, so it is a good preview of what you will build in the course project. The block diagram and all design files can be found in the repository, and the full documentation is available online.

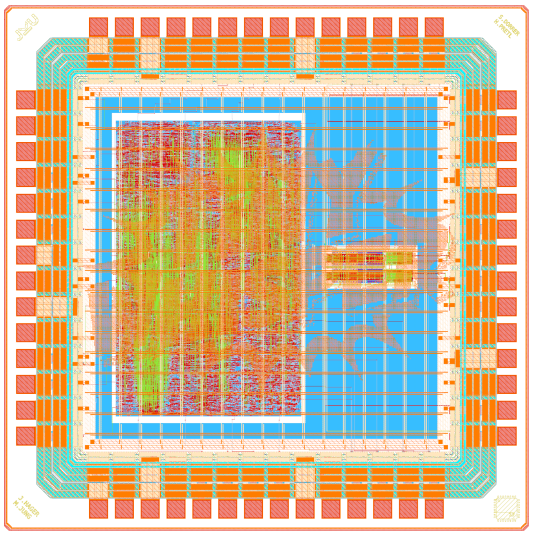
1.12.1 Weak Signal Propagation Reporter

TinyWhisper targets the **Weak Signal Propagation Reporter (WSPR)** protocol, a popular digital mode in amateur (ham) radio for propagation studies in the 80 m to 10 m bands. A WSPR transmitter sends a short message (callsign, location, and transmit power) using 4-

FSK modulation with a tone spacing of $\Delta f = 1.46\,\text{Hz}$ and a symbol duration of $T = 0.683\,\text{s}$. Receivers can decode signals down to $-31\,\text{dB}$ below the noise floor in a 2500 Hz bandwidth and report everything they receive to a central database via the Internet (WSPRnet map). On TinyWhisper, the modem part (coding, framing, and symbol mapping) is implemented in firmware written in C, which runs on the on-chip RISC-V CPU.

1.12.2 Chip Overview

Table 4 summarizes the key parameters of the chip, and Figure 3 shows renders of its layout. The custom logo on the topmost metal layer (Figure 3b) was generated with the tool ArtistIC of the PULP platform. TinyWhisper was taped out in the IHP open MPW run in March 2026 and, at the time of writing (October 2026), is still in fabrication.



(a) Final layout with the RISC-V CPU on the left and the IQ modulator on the right.



(b) TopMetal2 layer with the logo and metal fill.

Figure 3: Renders of the taped-out TinyWhisper chip (2000 µm × 2000 µm), taken from the TinyWhisper repository.

Table 4: Key parameters of the TinyWhisper chip

Parameter	Value
Technology	IHP SG13CMOS (130 nm CMOS), open PDK
Die area	2000 µm × 2000 µm (4 mm²)
Core area	1270 µm × 1270 µm (1.61 mm²)
Clock frequency	56 MHz
Core / I/O supply voltage	1.5 V / 3.3 V
Pads	56
Package	QFN-48 (8 pads bonded directly to package ground)

The **digital core** is built around an embedded RISC-V CPU implementing `rv32e_zicsr`, i.e., the 32-bit embedded base integer instruction set with 16 registers plus the extension for control and status registers. The CPU is a multi-cycle design without pipelining, running at 56 MHz, and accesses its SPI, I²C, UART, and GPIO peripherals as memory-mapped I/O. The I/

Q baseband signals are generated in hardware: a 30-bit iterative CORDIC computes sine and cosine with a frequency resolution of $\Delta f = f_{\mathrm{clk}}/2^{30} \approx 0.05\,\mathrm{Hz}$, and a single-bit second-order delta-sigma modulator (oversampling ratio up to 256) converts its output into a bitstream for the analog front-end. Both are configurable via software. Furthermore, the digital core divides the clock down to the 25% duty-cycle local oscillator (LO) signals of the mixers, which cover the 80 m, 40 m, 20 m, and 10 m bands.

The RISC-V CPU has already been proven in silicon: it was part of a multi-project student chip (with digital filters, mini games, and more) fabricated in the GlobalFoundries GF180MCU technology in the first wafer.space run (GitHub). On a breadboard, with the firmware supplied by an external microcontroller board and a discrete third-order multiple-feedback low-pass filter, the chip generates a clean baseband signal.

The **analog front-end** is an IQ modulator. Each of its two channels consists of a third-order multiple-feedback (MFB) Butterworth low-pass filter with a cutoff frequency of 400 kHz, built around a Barthélemy/Manfredini inverter-based operational transconductance amplifier (OTA), followed by a passive CMOS voltage-mode mixer with simple transmission-gate (TG) switches and a single-to-differential conversion circuit. Remarkably, the front-end consists only of inverter- and TG-based circuits. As an IQ transmitter, it supports not only WSPR but also other modes such as FT8, SSB, and CW.

Finally, even the **bonding diagram** for packaging is generated automatically: a Python script reads a configuration file with the pinout and placement, extracts the relevant layers (top metal and pads) from the layout, places the chip in the package, determines the pad centers, draws the bond wires, and renders the result as GDS. This is exactly what `make bondplan` of the AMS chip template does (Section 1.10.2).

> "The aha moment of seeing your circuit come to life will stay long after the equations are forgotten." — Peter Kinget

1.13 Tools Installation

You can either install the IIC-OSIC-TOOLS locally on your own laptop or use a workspace on a JKU server via your browser.

1.13.1 Local Installation

1. Install **Visual Studio Code** from https://code.visualstudio.com.
2. Run the interactive **installer**. It sets up Git, Docker (or Podman), WSL2 on Windows or XQuartz on macOS, and clones the IIC-OSIC-TOOLS repository (by default to `~/iic-osic-tools`). Every action asks for your confirmation first.

   On Windows, open PowerShell **as Administrator** and run:

   ```
   irm https://osic.tools/install.ps1 | iex
   ```

   On macOS and Linux, run:

   ```
   curl -fsSL https://osic.tools/install.sh | bash
   ```

3. Reboot your computer. On Windows and macOS, start Docker Desktop once afterwards.
4. Start the container from within the `iic-osic-tools` folder with `.\start_vnc.bat` (Windows) or `./start_vnc.sh` (macOS and Linux), and open http://localhost/?password=abc123 in your browser. The first start downloads the image (about 4 GB), so be patient. With Podman, the default port is 8080 instead of 80; the start script always prints the correct URL. On macOS and Linux, `./start_x.sh` alternatively shows the tools directly on your desktop via X11.
5. Store your designs in `~/eda/designs` on your laptop (`%USERPROFILE%\eda\designs` on Windows), which appears as `/foss/designs` inside the container. Only this folder persists when the container is removed or updated.

> **Tip**
>
> Piping a script from the Internet directly into a shell runs it without review. If you prefer to inspect the installer first, download it, read it, and run it locally, e.g., `curl -fsSLO https://osic.tools/install.sh && less install.sh && bash install.sh`.

1.13.2 Containerized On-Demand Workspace System (COWS)

If you cannot or do not want to install the tools locally, you can use the **Containerized On-Demand Workspace System (COWS)** instead, which provides a ready-to-use design environment in your browser:

1. Open COWS at https://jku128011.riic.uni-linz.ac.at/.
2. Log in with the credentials provided to you and change your password.
3. Create a workspace: go to *Workspaces*, click *Create workspace*, enter a *Workspace name*, and select the *KV VLSI Design 26/27* template.
4. Start the workspace; its *Status* should change to *Running*.
5. Open the workspace by clicking its name.

Enjoy your workspace, and happy designing!

2 From VHDL to SystemVerilog

> **Note**
>
> This chapter will be released later in the semester.

3 Modeling Guidelines for Synthesizable RTL

> **Note**
>
> This chapter will be released later in the semester.

4 Functional Verification with cocotb

> **Note**
>
> This chapter will be released later in the semester.

5 Sequential Elements and Static Timing Analysis

> **Note**
>
> This chapter will be released later in the semester.

6 The Synthesis Library

> **Note**
>
> This chapter will be released later in the semester.

7 Synthesis Optimizations

> **Note**
>
> This chapter will be released later in the semester.

8 Design for Test (DFT)

> **Note**
>
> This chapter will be released later in the semester.

9 Clock Domains and Metastability

> **Note**
>
> This chapter will be released later in the semester.

10 The RTL-to-GDSII Flow with LibreLane

> **Note**
>
> This chapter will be released later in the semester.

11 Capstone Project: A Small RISC-V SoC

> **Note**
>
> This chapter will be released later in the semester.

12 Appendix: SystemVerilog Cheat-Sheet

> **Note**
>
> This appendix will be released later in the semester.

13 Appendix: Tooling Cheat-Sheet

> **Note**
>
> This appendix will be released later in the semester.

14 Appendix: Sample Exam Questions

> **Note**
>
> This appendix will be released later in the semester.

Bibliography

[1] S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture, RISC-V Edition*. Morgan Kaufmann, 2021.

[2] N. H. E. Weste and D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Addison-Wesley, 2010.

[3] C. E. Cummings, *Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog*. Sunburst Design, 2008.

[4] Institute for Integrated Circuits, JKU, "IIC-OSIC-TOOLS: A Container with Open-Source IC Design Tools."

[5] S. Dorrer and H. Pretl, "An Open-Source Analog Mixed-Signal Chip Design Template & Tutorial for the ihp-sg13g2 Open-PDK." 2026.